PRODUCT REQUIREMENTS DOCUMENT

AI Video Agent

Prompt-to-Motion-Graphics Video Generator

Version:	1.0
Date:	March 2026
Agent Framework:	ReAct (Reasoning + Action)
LLM Provider:	Groq API — llama-3.3-70b-versatile
Rate Limit Mode:	Free Tier (6,000 TPM / 500k TPD)
Agent Dev:	Google Antigravity

1. Product Overview

The AI Video Agent converts a user's text goal and optional product image into a fully rendered motion-graphics product advertisement or demo video — automatically, using a ReAct agent powered by the Groq API.

Unlike diffusion-based text-to-video tools, this system uses an LLM-driven storyboard pipeline with programmatic React motion graphics. The system is specifically architected to operate within the Groq free tier rate limits.

Core Pipeline

```
User Prompt  →  ReAct Agent (Groq LLM)  →  Scene Storyboard
     →  Remotion Motion Graphics  →  ElevenLabs Voiceover  →  MP4
```

---

2. Goals & Objectives

- Convert any product description into a polished MP4 video with no manual steps
- Operate within Groq free tier limits: ≤ 6,000 TPM, ≤ 500,000 TPD
- Limit total Groq API calls to 3 per video run — all other steps are local computation
- Support text-only and text + image input modes
- Apply a robust ReAct agent architecture extensible with additional tools

---

3. Agent Framework — ReAct (Reasoning + Action)

3.1 Framework Comparison

Three agent frameworks were evaluated:

Framework	What it does	Tool Calls?	Decision
Chain of Thought (CoT)	LLM reasons step-by-step in text only — cannot call external APIs or tools	No	✕Rejected
ReAct (Reasoning + Action)	LLM alternates: Thought → Action (tool call) → Observation. Handles failures and replanning.	Yes	✓Selected
LangGraph / AutoGen	Multi-agent graph with parallel nodes, stateful edges, cross-agent communication	Yes	⚠Deferred

3.2 Why ReAct is Correct

- The pipeline must call external APIs (Groq, ElevenLabs, Remotion) — not just generate text
- Each tool result must be observed and validated before the next step proceeds
- Frame timings from Remotion must gate the ElevenLabs call — ReAct's Observe step enforces this
- CoT cannot dispatch any tool calls. LangGraph adds overhead for a single-agent sequential flow.

3.3 ReAct Loop

```
STEP 1  Thought → CALL feature_extractor → Observe { product, features[], tone,
duration }
STEP 2  Thought → CALL scene_planner     → Observe sceneManifest[]
STEP 3  Thought → CALL infographic_gen   → Observe chartAssets[]
STEP 4  Thought → CALL video_renderer    → Observe frameTimingManifest[]  ←
BLOCKING
STEP 5  Thought → CALL voice_generator   → Observe { sceneId, audioPath,
durationMs }[]
STEP 6  Thought → CALL video_renderer    → Observe mp4Path  →  FINISH

On 429 error: exponential backoff 2^n seconds (n=1,2,3). Max 3 retries per call.
```

---

4. Rate Limit Management (Groq Free Tier)

> ⚠ **This is the most critical engineering concern in the entire project.**
> The ReAct agent makes multiple Groq API calls per video within a short window.
> Without controls, the growing scratchpad alone will breach the 6,000 TPM limit.
> Every strategy below must be implemented before the agent is usable on the free tier.

4.1 Free Tier Limits — llama-3.3-70b-versatile

Limit Type	Free Tier Cap	Risk in This Project
Tokens per Minute (TPM)	6,000 tok/min	□ HIGH — multiple calls in ~30s window
Tokens per Day (TPD)	500,000 tok/day	□ LOW — each run uses ~2,650 tokens
Requests per Minute (RPM)	~30 req/min	□ LOW — max 3 Groq calls per run

4.2 Groq API Calls — Minimised to 3 Per Run

The most effective rate-limit control is reducing total Groq calls. The architecture achieves this by making only 3 LLM calls per video — all remaining steps are local computation with no API calls.

Step	Module	Output	Groq Call?	Notes
1	`promptParser`	`{ goal, hasImage, imagePath }`	No	Local parse only
2	`featureExtractor`	`{ product, features[], tone, duration }`	**Yes — Call 1**	~450 tokens
3	`scenePlanner`	`sceneManifest[]`	**Yes — Call 2**	~600 tokens
4	`infographicGenerator`	`chartAssets[]`	**Yes — Call 3**	~450 tokens
5	`videoRenderer (timings_only)`	`frameTimingManifest[]`	No	BLOCKING step
6	`scriptGenerator`	`narration scripts`	No	Built locally
7	`voiceGenerator (ElevenLabs)`	`{ audioPath, durationMs }[]`	No	Non-Groq API
8	`videoComposer (ffmpeg)`	`mp4Path`	No	Local render

> ✅**Only 3 of 8 steps call Groq. All remaining steps are local — zero token cost.**

4.3 Token Budget Per Groq Call

Each Groq call is capped via the max_tokens parameter. The token estimator pre-checks input length before every call and blocks it if input exceeds 900 tokens.

Call	Purpose	Max Input (tok)	Max Output (tok)	Total Cap	Status
1	Feature Extraction	300	150	450	□ Safe
2	Scene Planning	350	250	600	□ Safe
3	Infographic Spec	300	150	450	□ Safe
—	Subtotal	950	550	~1,500	
—	+ System prompt overhead (~60/call × 3)	—	—	~180	
—	TOTAL PER RUN	—	—	~1,680	□ Well under 6,000 TPM

4.4 Scratchpad Truncation — Critical Implementation Rule

> ⚠ **This is the single largest token risk in the entire system.**

> Naive ReAct sends the full conversation history (all Thoughts + Observations) with every call. By Step 6 this alone can exceed 3,000 tokens of input — blowing the 6,000 TPM budget.

Solution: stateManager.js implements a sliding scratchpad window. Each Groq call receives only:

- Compressed system prompt — fixed ~100 tokens, written once, never changes
- Current step instruction — ~50–100 tokens describing exactly what to do this step
- Most recent observation only — the output from the immediately preceding tool call

All previous observations are stored in stateManager for reference and retry purposes, but are never sent back to Groq.

> Naive (DON'T DO): [step1 thought] + [step1 obs] + [step2 thought] + [step2 obs] + ...
> grows by ~200–400 tokens per step → 1,500+ tokens by Step 6
>
> Correct: [system prompt ~100 tok] + [step instruction ~80 tok] + [last obs ~150 tok]
> fixed ~330 tokens input per call regardless of how many steps have run

4.5 Prompt Engineering for Token Efficiency

Every prompt sent to Groq is designed for minimum tokens while preserving output accuracy.

System Prompt — Fixed, ~100 tokens

```
You are a video storyboard agent. Reply ONLY in JSON. No prose. No markdown.
Extract features, plan scenes, or validate results as instructed each step.
```

Step 2 — Scene Planner User Prompt Example (~70 tokens in, ~120 tokens out)

```
Input: {product:"earbuds", features:["noise cancellation","8hr battery"],
        tone:"energetic", duration:15}
Plan 5 scenes. Return: [{id,type,content,durationSec}]
```

Output format	JSON only — no explanation, no prose, no markdown wrappers
Field names	Short keys: id, type, content, dur (not durationSeconds)
Feature list	Max 4 features extracted — no lengthy descriptions
Scene content	Max 6 words per scene content field
Tone values	Single word only: premium, energetic, friendly, corporate, neutral

4.6 Rate Limit Controls in groqClient.js

groqClient.js wraps every Groq API call and enforces all of the following controls:

Control	Implementation Detail

max_tokens cap	Every API call sets max_tokens to the per-step budget (150–250 output tokens). Groq stops generating at this limit even if the response is incomplete.
Inter-step delay	1 second sleep between consecutive Groq calls (setTimeout 1000ms). Reduces burst token rate even when calls complete quickly.
Token estimator	tokenEstimator.js counts input tokens using char/4 approximation before every call. If estimated input exceeds 900 tokens, the call is blocked and an error is logged.
429 exponential backoff	On HTTP 429 response: wait 2^n seconds where n = retry attempt number (2s, 4s, 8s). Max 3 retries. Uses the retry-after header value if present in the 429 response.
Model fallback	If all 3 retries on llama-3.3-70b-versatile are exhausted, groqClient.js switches to llama-3.1-8b-instant (which has ~30,000 TPM on free tier) for the remainder of that session.
Daily token tracker	stateManager.js accumulates total tokens used per day (input + output). Logs a warning at 400,000 tokens. Returns an error at 490,000 tokens to preserve a safety buffer.
Response headers	Reads x-ratelimit-remaining-tokens and x-ratelimit-reset-tokens from Groq response headers. If remaining < 1,500, inserts an additional delay before the next call.

4.7 Model Fallback Strategy

Primary model	llama-3.3-70b-versatile — ~6,000 TPM on free tier. High quality. Used for all calls.
Fallback model	llama-3.1-8b-instant — ~30,000 TPM on free tier. Good quality. Auto-switch on 429 exhaustion.
Fallback trigger	Activated by groqClient.js after 3 consecutive 429 errors on the primary model
Fallback scope	Applies to the rest of the current session only. Next session resets to primary.
Output compatibility	Both models return JSON. Schema is identical. No code changes required at call sites.

---

5. Feature Specifications

Feature 1 — Prompt-to-Video Generation

- User provides a natural language goal — the ReAct agent handles everything else
- Total Groq token consumption per run: ~1,680 tokens (leaves 4,320 token buffer in 6,000 TPM cap)
- Full pipeline from prompt to rendered MP4 with zero manual steps

Feature 2 — Multimodal Input Support

Text Only	Agent uses text animations, infographics, stock visual fallbacks
Text + Image	Product showcase scene with uploaded image. Fallback to text if image fails to load

Feature 3 — AI Scene Planning (Groq API · llama-3.3-70b-versatile)

- Groq LLM tool call returns structured JSON scene plan — JSON-only output, no prose
- Compact prompts keep this call under 600 tokens total (350 in + 250 out)
- Default flow: Title → Product Showcase → Feature Highlight → Infographic → CTA
- Scene count and order adapt dynamically based on extracted features

Feature 4 — Motion Graphics Generation (Remotion)

- React components render each scene — no Groq call, zero token cost
- Animations: fade-in, zoom, slide, kinetic typography, spring transitions, chart reveals
- Output: frameTimingManifest[] — required input for ElevenLabs voice sync

Feature 5 — Infographic Generation

- Chart data generated by Groq call 3 — capped at 450 tokens total
- Chart types: animated bar chart, pie chart, stat badge, icon strip

Feature 6 — AI Voiceover (ElevenLabs API)

- Narration scripts built locally from sceneManifest — no Groq call for script writing
- ElevenLabs API call is independent of Groq rate limits
- Audio must be generated after frameTimingManifest — enforced by ReAct Observe step

Feature 7 — Automated Video Rendering (Remotion CLI + ffmpeg)

- Remotion CLI renders all scenes headless at 30fps, 1920x1080 — no Groq call
- ffmpeg muxes video + AAC audio → final H.264/AAC MP4
- Two render modes: timings_only (get frame map) and full_render (write MP4)
- ffmpeg is a required runtime dependency — must be installed on the render host

---

6. High-Level Architecture

LLM Backbone	Groq API — llama-3.3-70b-versatile (fallback: llama-3.1-8b-instant)
Agent Pattern	ReAct — Thought → Action → Observe
Groq Calls / Run	3 total (feature extract, scene plan, infographic spec)
Token Budget / Run	~1,680 tokens total (well under 6,000 TPM)
Rate Guard	groqClient.js — token cap, inter-step delay, backoff, fallback, header monitor
Sequencing Constraint	video_renderer (timings_only) must complete before voice_generator
Tool Dispatch	toolRegistry.js
Session State	stateManager.js — sliding scratchpad + daily token tracker

```
Pipeline:
  User Input  →  ReAct Orchestrator (groqClient.js rate guard)
    ↓ Groq call 1: feature_extractor  →  { product, features[], tone, duration }
    ↓ Groq call 2: scene_planner      →  sceneManifest[]
    ↓ Groq call 3: infographic_gen    →  chartAssets[]
    ↓ LOCAL: video_renderer (timings_only)  →  frameTimingManifest[]  ← BLOCKING
    ↓ LOCAL: scriptGenerator          →  narration scripts
    ↓ ElevenLabs: voice_generator     →  audioPaths[]
    ↓ LOCAL: videoComposer (ffmpeg)   →  MP4
```

---

7. Low-Level Architecture

Layer 1 — UI Layer (React + TailwindCSS)

PromptInput.jsx	Text goal input. Character limit: 500. Submit triggers agent.
ImageUpload.jsx	Drag-and-drop. Accepts PNG/JPG/WebP. Max 5 MB.
PreviewPanel.jsx	Shows current ReAct step, token usage indicator, progress bar, download button.

Layer 2 — ReAct Agent Layer (Groq API)

agentOrchestrator.js — ReAct loop controller

- Thought → Action → Observe cycle, max 10 steps
- Enforces 1 second inter-step delay between every Groq call
- Dispatches tools via toolRegistry.js
- Sends ONLY last observation to Groq — not full history (scratchpad truncation)
- Retries ≤ 3 times on tool failure, then structured abort

groqClient.js — Rate-limit wrapper (NEW MODULE)

- Sets max_tokens per call according to per-step token budget
- Calls tokenEstimator.js before every request — blocks if input > 900 tokens
- Handles 429: exponential backoff 2s → 4s → 8s, reads retry-after header if present
- After 3 failed retries: switches to llama-3.1-8b-instant for rest of session
- Reads x-ratelimit-remaining-tokens header — adds extra delay if < 1,500 remaining

tokenEstimator.js — Pre-call guard (NEW MODULE)

- Estimates tokens using character count / 4 approximation (rough but sufficient as a guard)
- Accepts: systemPrompt string + userPrompt string
- Returns: { estimatedTokens, safe: boolean }
- Blocks the call and logs a warning if estimatedTokens > 900

stateManager.js — Session memory + token tracker

- Sliding scratchpad: stores all tool results but only exposes last observation to Groq
- Token tracker: accumulates input + output tokens per call; warns at 400k/day; hard stops at 490k/day
- Stores: currentStep, toolResults (keyed), retryCount per tool, sessionId (UUID)

promptParser.js	Local only. OUT: { goal, hasImage, imagePath }. No Groq call.
featureExtractor.js	Groq call 1. Max 300 input / 150 output tokens. OUT: { product, features[], tone, duration }
scenePlanner.js	Groq call 2. Max 350 input / 250 output tokens. OUT: sceneManifest[]

toolRegistry.js	Registers all tools: feature_extractor, scene_planner, infographic_generator, voice_generator, video_renderer

Layer 3 — Media Generation Layer (Remotion + React)

No Groq calls in this layer. All rendering is local computation.

TitleScene.jsx	Animated title, fade-in, gradient background
ProductScene.jsx	Product image zoom + reveal. Text fallback if no image.
FeatureScene.jsx	Kinetic typography, slide-in animation
InfographicScene.jsx	Animated bar, pie, stat badge charts
CTAScene.jsx	Pulse ring + highlight animation
assetManager.js	Image load, icon bank, stock fallbacks

Output contract: frameTimingManifest[] — { sceneId, startFrame, endFrame, durationSec, fps:30 }

> ⚠ **frameTimingManifest is a required input for Layer 4. Layer 4 cannot begin until this is emitted.**

Layer 4 — Voice Layer (ElevenLabs API)

No Groq calls in this layer. Scripts are built locally from sceneManifest.

scriptGenerator.js	Builds per-scene narration locally from sceneManifest + frameTimingManifest. Zero token cost.
toneMatcher.js	Maps tone → ElevenLabs voice ID: premium→Rachel, energetic→Adam, friendly→Elli, corporate→Daniel, neutral→Callum
audioSynchronizer.js	Trims audio if longer than scene. Pads with silence if shorter. OUT: { sceneId, audioPath, durationMs }[]

Layer 5 — Rendering Layer (Remotion CLI + ffmpeg)

No Groq calls. All local computation.

sceneSequencer.js	Assigns frame ranges, calculates total duration, binds asset paths
videoRenderer.js	Remotion CLI. Modes: timings_only (emit manifest) / full_render (write MP4). 30fps · 1920×1080.
mp4Exporter.js	ffmpeg mux: PNG frame sequence + AAC audio → H.264/AAC MP4

> **⚠ Runtime dependency: ffmpeg must be installed on the server or container running this application.**

---

8. Critical Execution Order

Step	Module	Output	Groq Call?	Notes
1	`promptParser`	`{ goal, hasImage, imagePath }`	No	Local only
2	`featureExtractor`	`{ product, features[], tone, duration }`	**Yes — Call 1**	≤450 tokens
3	`scenePlanner`	`sceneManifest[]`	**Yes — Call 2**	≤600 tokens
4	`infographicGenerator`	`chartAssets[]`	**Yes — Call 3**	≤450 tokens
5	`videoRenderer (timings_only)`	`frameTimingManifest[]`	No	**⚠BLOCKING**
6	`scriptGenerator`	`narration scripts built locally`	No	Zero token cost
7	`voiceGenerator (ElevenLabs)`	`{ audioPath, durationMs }[]`	No	Non-Groq API
8	`videoComposer (ffmpeg)`	`mp4Path`	No	Local render

9. Technology Stack

Layer	Technology	Detail
Frontend UI	React + TailwindCSS	User input form, image upload, preview panel, token usage indicator
Motion Graphics	Remotion	React-based frame-by-frame scene components
Agent Framework	ReAct (agentOrchestrator.js)	Thought → Action → Observe, max 10 steps, 1s inter-step delay
LLM Inference	Groq API	Primary: llama-3.3-70b-versatile · Fallback: llama-3.1-8b-instant
Rate Limit Guard	groqClient.js	Token cap, exponential backoff, fallback, header-based throttle
Token Guard	tokenEstimator.js	Pre-call token count guard — blocks if input > 900 tokens
Voiceover	ElevenLabs API	Per-scene narration, five voice tones
Agent Dev	Google Antigravity	AI-assisted module code generation
Video Encoding	ffmpeg	Mux Remotion video frames with AAC audio
Rendering	Remotion CLI (headless)	PNG frame sequence + scene composition

10. Google Antigravity Development Workflow

Each module is built by giving Antigravity a prompt that specifies: input schema, output schema, error behaviour, and token constraints where relevant.

```
Describe module (input schema, output schema, error rules, token constraints)
                    ↓
      Antigravity generates module code
                    ↓
      Developer validates logic + token behaviour
                    ↓
      Module integrated into agent pipeline
```

Recommended Build Order

Sprint 1	UI: PromptInput, ImageUpload, PreviewPanel

Sprint 2	groqClient.js, tokenEstimator.js — build rate controls first before any agent code
Sprint 3	agentOrchestrator.js, toolRegistry.js, stateManager.js (with sliding scratchpad)
Sprint 4	promptParser.js, featureExtractor.js, scenePlanner.js
Sprint 5	Scene components: TitleScene, ProductScene, FeatureScene, InfographicScene, CTAScene
Sprint 6	Voice layer: scriptGenerator, toneMatcher, audioSynchronizer
Sprint 7	Rendering: sceneSequencer, videoRenderer, mp4Exporter

---

11. Project Structure

```
ai-video-agent/
├── src/
│   ├── agent/
│   │   ├── agentOrchestrator.js   ← ReAct core + 1s inter-step delay
│   │   ├── groqClient.js          ← Rate-limit wrapper (NEW)
│   │   ├── tokenEstimator.js      ← Pre-call token guard (NEW)
│   │   ├── toolRegistry.js
│   │   ├── stateManager.js        ← Sliding scratchpad + daily token tracker
│   │   ├── promptParser.js
│   │   ├── featureExtractor.js
│   │   └── scenePlanner.js
│   ├── scenes/
│   │   ├── TitleScene.jsx
│   │   ├── ProductScene.jsx
│   │   ├── FeatureScene.jsx
│   │   ├── InfographicScene.jsx
│   │   ├── CTAScene.jsx
│   │   └── assetManager.js
│   ├── voice/
│   │   ├── scriptGenerator.js
│   │   ├── toneMatcher.js
│   │   └── audioSynchronizer.js
│   ├── composer/
│   │   ├── sceneSequencer.js
│   │   ├── videoRenderer.js
│   │   └── mp4Exporter.js
│   └── ui/
│       ├── PromptInput.jsx
│       ├── ImageUpload.jsx
│       └── PreviewPanel.jsx
├── public/
├── README.md
├── PRD.docx
└── package.json
```